

Document 06 — Implementation Plan

Step-by-Step Build Sequence for NowWise

Product Name

NowWise

Internal Platform

NowWise OS

Product Type

Telegram-first, voice-first AI lifestyle operating system for food, grocery, restaurant, product, habit, and behavior intelligence.

Purpose of This Document

This implementation plan tells Claude/Codex exactly what to build first, second, and third.

The goal is to prevent random implementation order.

Do not start with UI polish.

Do not start with mobile app.

Do not start with barcode scanner.

Do not start with insurance analytics.

Do not start with full OpenWhisper desktop integration.

Build the foundation first.

1. Final Build Strategy

NowWise should be built in this order:

1. Repo and environment setup
2. Database schema and migrations
3. Admin auth and protected dashboard shell
4. Telegram bot foundation
5. Universal event engine

6. Text logging
7. Voice processing pipeline
8. Intent classifier
9. Food logging
10. Product lookup
11. Reminder engine
12. Weekly report engine
13. Admin review queue
14. Product compliance portal
15. Privacy controls
16. UI polish
17. Testing
18. Deployment

The reason is simple:

Every feature depends on the event engine, user model, and schema. Build those first.

2. Technical Stack Locked

Layer	Stack
Frontend	Next.js + TypeScript
Styling	Tailwind CSS
UI Components	shadcn/ui + Radix UI
Database	Supabase Postgres
Auth	Supabase Auth for admins
User Identity	Telegram user ID and chat ID
Bot	Telegram Bot API
Voice Service	Python FastAPI
Audio Conversion	FFmpeg
Local ASR	OpenWhisper-inspired Whisper/Parakeet service
AI Providers	OpenAI / Claude / Gemini via provider abstraction
Queue	Redis + BullMQ or Supabase queue
Hosting	Vercel + Supabase + Railway/Fly/Render workers
Monitoring	Sentry + structured logs

3. Phase 1 — Project Setup

Goal

Create the project foundation, repo structure, dependency setup, env files, and coding conventions.

Tasks

1.1 Initialize Monorepo

Create:

```
nowwise/  
  apps/  
    web/  
  services/  
    telegram-bot/  
    voice-service/  
  workers/  
  packages/  
    db/  
    ai/  
    voice/  
    shared/  
  docs/
```

1.2 Create Next.js App

Inside:

```
apps/web
```

Use:

```
Next.js  
TypeScript  
Tailwind CSS  
shadcn/ui
```

```
ESLint
Prettier
```

1.3 Create Python Voice Service

Inside:

```
services/voice-service
```

Use:

```
FastAPI
Uvicorn
Pydantic
python-multipart
FFmpeg integration
```

1.4 Create Worker Service

Inside:

```
services/workers
```

Workers needed later:

```
processVoiceNote
processPhoto
sendReminder
generateWeeklyReport
detectPatterns
cleanupRawAudio
```

1.5 Environment Files

Create:

```
.env.example  
.env.local  
.env.staging  
.env.production
```

Do not commit real secrets.

1.6 Add Required Environment Variables

```
NEXT_PUBLIC_APP_URL  
NODE_ENV  
APP_ENV  
  
NEXT_PUBLIC_SUPABASE_URL  
NEXT_PUBLIC_SUPABASE_ANON_KEY  
SUPABASE_SERVICE_ROLE_KEY  
SUPABASE_DB_URL  
  
TELEGRAM_BOT_TOKEN  
TELEGRAM_WEBHOOK_SECRET  
TELEGRAM_WEBHOOK_URL  
  
OPENAI_API_KEY  
ANTHROPIC_API_KEY  
GOOGLE_GENERATIVE_AI_API_KEY  
GROQ_API_KEY  
  
VOICE_PROVIDER_DEFAULT  
LOCAL_ASR_BASE_URL  
LOCAL_ASR_API_KEY  
OPENAI_TRANSCRIPTION_MODEL  
GROQ_TRANSCRIPTION_MODEL  
DEEPGRAM_API_KEY  
WISPR_FLOW_API_KEY  
  
SUPABASE_AUDIO_BUCKET  
SUPABASE_PHOTO_BUCKET  
SUPABASE_RECEIPT_BUCKET  
  
REDIS_URL  
QUEUE_PREFIX  
  
SENTRY_DSN  
LOG_LEVEL
```

```
ADMIN_ALLOWED_EMAILS
ADMIN_INVITE_SECRET
```

Done Criteria

Phase 1 is complete when:

- Repo structure exists.
- Next.js app runs locally.
- Python voice service runs locally.
- `.env.example` includes all required variables.
- TypeScript, ESLint, Prettier, and Tailwind are configured.
- Basic health check endpoint works.
- README includes local setup steps.

4. Phase 2 — Database Schema and Migrations

Goal

Create the Supabase schema before building features.

Tasks

2.1 Enable Extensions

Create migration:

```
001_enable_extensions.sql
```

Enable:

```
pgcrypto
uuid-oss
pg_trgm
vector
```

2.2 Create Enums

Create migration:

```
002_create_enums.sql
```

Enums:

```
user_role  
event_type  
event_source  
event_status  
reminder_action  
product_decision  
review_status
```

2.3 Create Core Tables

Create migrations for:

```
users  
user_profiles  
user_events  
voice_events  
food_logs  
water_logs  
products  
product_variants  
product_logs  
product_corrections  
restaurant_logs  
grocery_receipts  
reminders  
reminder_events  
behavior_patterns  
weekly_reports  
admin_reviews  
consent_settings  
privacy_requests  
audit_logs  
voice_provider_metrics
```

2.4 Create Indexes

Add indexes for:

```
telegram_user_id  
telegram_chat_id  
user_id + occurred_at  
event_type  
event_status  
voice provider  
product normalized_name trigram  
barcode  
reminder next_run_at  
review status  
weekly report user/week  
privacy request status
```

2.5 Create RLS Policies

Enable RLS on sensitive tables.

Tables requiring RLS:

```
users  
user_profiles  
user_events  
voice_events  
food_logs  
product_logs  
reminders  
reminder_events  
behavior_patterns  
weekly_reports  
consent_settings  
privacy_requests
```

2.6 Create Storage Buckets

Buckets:

```
voice-temp  
food-photos  
receipt-images  
menu-images  
product-images  
exports
```

All buckets private by default.

2.7 Seed Test Data

Seed:

- 3 test Telegram users
 - 2 admin users
 - 10 sample products
 - 5 food logs
 - 3 reminders
 - 3 voice events
 - 3 admin reviews
-

Done Criteria

Phase 2 is complete when:

- All migrations run cleanly on local/staging Supabase.
 - All tables exist.
 - Indexes exist.
 - RLS is enabled.
 - Storage buckets exist.
 - Seed data appears correctly.
 - Admin user can be created.
 - No service role key is exposed to frontend.
-

5. Phase 3 — Admin Authentication

Goal

Create protected admin access before building dashboard operations.

Tasks

3.1 Build Admin Login Page

Route:

```
/admin/login
```

Fields:

```
Email  
Password
```

3.2 Supabase Auth Integration

Implement:

- Login
- Logout
- Session persistence
- Auth callback if needed
- Protected route middleware

3.3 Admin Role Check

After login:

1. Get Supabase session.
2. Find matching user by email.
3. Check role.
4. Allow only admin roles.

Allowed roles:

```
super_admin  
admin  
reviewer  
product_admin  
support  
read_only
```

3.4 Protected Admin Layout

Create:

```
/admin/dashboard  
/admin/users  
/admin/events  
/admin/voice-events  
/admin/products  
/admin/product-reviews  
/admin/reminders  
/admin/weekly-reports  
/admin/privacy-requests  
/admin/settings
```

Done Criteria

Phase 3 is complete when:

- Admin can login.
- Admin can logout.
- Unauthorized users are blocked.
- Protected admin routes redirect to `/admin/login`.
- Admin role is loaded correctly.
- Sidebar layout appears after login.

6. Phase 4 — Telegram Bot Foundation

Goal

Build the primary user interface.

Tasks

4.1 Create Telegram Bot

Create bot using BotFather.

Set:

```
Bot name
Bot username
Bot token
Command menu
Webhook URL
```

4.2 Webhook Endpoint

Route:

```
POST /api/telegram/webhook
```

Must:

- Verify webhook secret.
- Receive Telegram updates.
- Parse message type.
- Return fast response.
- Queue heavy jobs.

4.3 Telegram User Creation

When a new Telegram user messages:

1. Check `users.telegram_user_id`.
2. If not found, create user.
3. Create `user_profiles`.
4. Create `consent_settings`.
5. Send onboarding message.

4.4 Bot Commands

Implement:

```
/start
/help
/log_food
/check_product
/set_reminder
/my_week
```

```
/my_goal  
/privacy
```

4.5 Main Menu

Buttons:

 Log food
 Check product
 Set reminder
 My week
 My goal
 Privacy

Done Criteria

Phase 4 is complete when:

- Telegram bot receives messages.
- New users are created.
- Existing users are recognized.
- `/start` works.
- Main menu works.
- User records are stored in Supabase.
- Webhook responds quickly.

7. Phase 5 — Onboarding Flow

Goal

Collect only necessary user context.

Tasks

5.1 Welcome Message

Bot says:

Hi, I'm NowWise.
I help you make better food, grocery, restaurant, and habit decisions.
You can speak, type, or send photos.

5.2 Goal Selection

Buttons:

Better food habits
Digestion
Sugar control
Weight control
Energy
Custom

Save to:

`user_profiles.primary_goal`

5.3 Language Preference

Buttons:

English
Hinglish
Hindi
Telugu

Save to:

`users.language_preference`

5.4 Timezone Confirmation

Default:

Asia/Kolkata

Bot asks:

I'll use Asia/Kolkata for your reminders and food timeline. Is that correct?

5.5 First Action

Buttons:

-  Log food
-  Check product
-  Set reminder
-  See example weekly report

Done Criteria

Phase 5 is complete when:

- New users complete onboarding.
- Goal is saved.
- Language is saved.
- Timezone is saved.
- `onboarded_at` is populated.
- Existing users skip onboarding.

8. Phase 6 — Universal Event Engine

Goal

Every meaningful interaction becomes a timeline event.

Tasks

6.1 Create Event Service

Create:

```
packages/db/events.ts
```

Functions:

```
createUserEvent()  
updateUserEventStatus()  
getUserTimeline()  
attachDomainRecord()  
markNeedsReview()  
markFailed()
```

6.2 Event Types

Support:

```
food_log  
product_lookup  
voice_note  
photo_upload  
receipt_scan  
restaurant_menu  
reminder_created  
reminder_checkin  
water_log  
daily_reflection  
weekly_report  
correction  
privacy_request  
unknown
```

6.3 Auto Timestamp

Every event must store:

```
occurred_at  
timezone  
created_at
```

Do not ask user to manually pick date/time unless they mention past/future time.

6.4 Meal Inference

Create helper:

```
inferMealType(occurred_at, timezone)
```

Default logic:

```
5 AM-10:30 AM: breakfast  
10:30 AM-12 PM: snack/early lunch clarification  
12 PM-3 PM: lunch  
3 PM-6 PM: snack  
6 PM-10:30 PM: dinner  
10:30 PM-5 AM: late-night
```

Done Criteria

Phase 6 is complete when:

- Text message creates `user_event` .
- Voice message creates `user_event` .
- Reminder request creates `user_event` .
- Product lookup creates `user_event` .
- Event timeline can be fetched by user/date.
- Meal type is inferred automatically.

9. Phase 7 — Text Food Logging

Goal

Allow the user to type food and get useful response.

Tasks

7.1 Text Intent Detection

When user types:

```
I ate rice dal curd
```

Detect:

```
intent = food_log
```

7.2 Food Entity Extraction

Extract:

```
foods  
contains_sweet  
contains_fried  
contains_sugary_drink  
protein_quality  
vegetable_presence
```

7.3 Create Food Log

Insert into:

```
food_logs
```

Link to:

```
user_events.id
```

7.4 Telegram Response

Example:

```
Logged as lunch at 1:18 PM.  
  
You had rice, dal, and curd.  
Add protein later today if possible.
```

Buttons:

Correct
Add more
View today

Done Criteria

Phase 7 is complete when:

- User can type meal.
- Food log is created.
- Event is linked.
- Telegram reply is generated.
- User can correct meal type/basic food.

10. Phase 8 — Voice Processing Pipeline

Goal

Let user send Telegram voice notes.

Tasks

8.1 Voice Note Ingestion

When Telegram sends voice note:

1. Create `user_event`.
2. Create `voice_event`.
3. Queue `processVoiceNote`.

8.2 Audio Download

Worker downloads file using Telegram API.

Save temporarily to:

```
voice-temp/{user_id}/{voice_event_id}.ogg
```

8.3 Audio Conversion

Use FFmpeg to convert:

```
OGG/OPUS → WAV
```

8.4 Local ASR Endpoint

Build Python FastAPI endpoint:

```
POST /v1/audio/transcriptions
```

Input:

```
multipart/form-data file
model
language
prompt
```

Output:

```
{
  "text": "I ate rice, dal, curd, and one sweet."
}
```

8.5 Voice Provider Router

Implement providers:

```
local_openwhisper
local_whisper
parakeet
openai
groq
```

```
deepgram
wispr_flow_optional
```

8.6 Cloud Fallback

If local ASR fails:

1. Try cloud fallback.
 2. Mark provider used.
 3. Store latency.
 4. Continue processing.
-

8.7 Delete Raw Audio

After transcription:

- Delete raw audio unless user consent allows storage.
 - Update `raw_audio_deleted_at`.
-

8.8 Process Transcript

Send transcript to intent classifier.

Done Criteria

Phase 8 is complete when:

- User sends Telegram voice note.
 - Audio is downloaded.
 - Audio is converted.
 - Transcript is generated.
 - Transcript is stored.
 - Raw audio is deleted.
 - Failed voice notes create admin review.
 - Voice response works end-to-end.
-

11. Phase 9 — Intent Classifier and Entity Extractor

Goal

Convert messages/transcripts into actions.

Tasks

9.1 Build AI Provider Router

Create:

```
packages/ai/provider-router.ts
```

Support:

```
OpenAI  
Claude  
Gemini  
Fallback mock provider for tests
```

9.2 Supported Intents

Detect:

```
food_log  
product_lookup  
reminder_creation  
habit_checkin  
water_log  
grocery_basket  
restaurant_decision  
correction  
weekly_summary_request  
privacy_request  
unknown
```

9.3 Entity Extraction

Extract:

Intent	Entities
food_log	foods, quantity, meal time
product_lookup	brand, product name, variant
reminder_creation	habit, time, frequency
habit_checkin	habit, action
grocery_basket	item list
restaurant_decision	dish options
correction	target event, correction type

9.4 Confidence Scoring

If confidence below threshold:

confidence < 0.70

Create admin review.

If user clarification can solve it, ask one question.

Done Criteria

Phase 9 is complete when:

- Text and voice both pass through same intent layer.
- Food intent works.
- Product intent works.
- Reminder intent works.
- Unknown intent has safe fallback.
- Low-confidence items route to review.

12. Phase 10 — Product Lookup and Product Graph

Goal

Let user ask “Can I buy/eat this?”

Tasks

10.1 Product Search

Search by:

```
brand
product_name
normalized_name
barcode later
```

Use trigram fuzzy search.

10.2 Product Matching

If one strong match:

- Use product directly.

If multiple matches:

Bot asks:

```
Which one is it?
```

Buttons:

```
Sunflower Seeds
Pumpkin Seeds
Mixed Seeds
None of these
```

10.3 Product Decision Engine

Return:

```
can_have  
occasionally  
avoid_today  
ask_clarification  
unknown
```

Use:

- Product data
 - User goal
 - Today's timeline
 - Weekly patterns
 - Confidence
-

10.4 Product Response

Example:

```
I found Tony Garden Sunflower Seeds.
```

```
Decision: Okay occasionally.
```

```
If this is salted, keep it to a small handful. Unsalted is better today because  
you already had salty snacks twice this week.
```

Buttons:

```
Salted  
Unsalted  
Wrong product  
Show alternative
```

Done Criteria

Phase 10 is complete when:

- User can type product.
 - User can speak product.
 - Product match works.
 - Product log is created.
 - Decision is generated.
 - Wrong product flow creates review.
 - Admin can see product lookup.
-

13. Phase 11 — Reminder Engine

Goal

Create smart habit reminders.

Tasks

11.1 Reminder Parser

Parse:

Remind me daily at 9:30 PM to take Isabgol.

Extract:

```
habit_name = Isabgol
frequency = daily
time = 21:30
timezone = Asia/Kolkata
```

11.2 Create Reminder

Insert into:

reminders

Create event:

```
user_events.event_type = reminder_created
```

11.3 Reminder Scheduler

Worker checks:

```
active = true  
next_run_at <= now()
```

Then sends Telegram reminder.

11.4 Reminder Buttons

Buttons:

```
Taken  
Later  
Skip
```

11.5 Reminder Actions

Create:

```
reminder_events
```

If Taken:

```
user_action = taken
```

If Later:

Ask:

```
15 min  
30 min  
1 hour  
Tomorrow
```

If Skip:

```
user_action = skip
```

11.6 Smart Skip Logic

If user already says:

```
I took Isabgol
```

before reminder, mark completed and do not send duplicate reminder.

Done Criteria

Phase 11 is complete when:

- User can create reminder by text.
 - User can create reminder by voice.
 - Reminder sends at correct time.
 - Taken/Later/Skip works.
 - Reminder event is stored.
 - Weekly report can read reminder adherence.
-

14. Phase 12 — Weekly Report Engine

Goal

Turn logs into weekly behavior story.

Tasks

12.1 Weekly Report Worker

Run weekly.

For each active user:

- Fetch food logs.
 - Fetch product logs.
 - Fetch reminder events.
 - Fetch behavior patterns.
 - Generate summary.
-

12.2 Pattern Detection

Detect:

```
sweets_high_frequency  
high_sodium_snacks  
late_night_eating  
low_protein  
low_vegetables  
low_hydration  
habit_skipping
```

12.3 Report Structure

Weekly report contains:

```
Food logs count  
Product checks count  
Habit completion  
Sweets pattern  
Vegetable/protein pattern  
Best improvement  
Next week focus
```

12.4 Telegram Report

Example:

Your NowWise Week

Food logs: 18

Product checks: 6

Isabgol: 4/7 days

Sweets: 5 days

Vegetables: 2 days

Best improvement:

You logged dinner consistently.

Next week focus:

Reduce sweets to 3 days and add vegetables at dinner.

Done Criteria

Phase 12 is complete when:

- Weekly report can be generated.
- Report is stored in `weekly_reports`.
- Report can be sent via Telegram.
- User can request `/my_week`.
- Empty state works if not enough data.

15. Phase 13 — Admin Dashboard

Goal

Build operations dashboard for team.

Tasks

13.1 Dashboard Home

Route:

/admin/dashboard

Cards:

Active users
Logs today
Voice notes today
Product lookups
Reminders sent
Reminder completion rate
Low-confidence events
Failed transcriptions
Weekly reports sent
Raw audio deletion success

13.2 Users Page

Route:

/admin/users

Show:

Name
Telegram username
Goal
Language
Timezone
Last seen
Events count

13.3 Events Page

Route:

/admin/events

Filters:

Date range
User
Event type
Source
Status
Confidence

13.4 Voice Events Page

Route:

/admin/voice-events

Show:

Transcript
Provider
Language
Confidence
Latency
Status
Raw audio deleted?

13.5 Review Drawers

Clicking rows opens right-side drawer.

Actions:

Approve
Edit
Reject
Send to review

Done Criteria

Phase 13 is complete when:

- Admin dashboard loads.
 - Users/events/voice events are visible.
 - Filters work.
 - Drawers work.
 - Admin can review low-confidence items.
 - Audit logs are created for sensitive actions.
-

16. Phase 14 — Product Compliance Portal

Goal

Let admin maintain product intelligence database.

Tasks

14.1 Product Database Page

Route:

```
/admin/products
```

Features:

```
Search
Filter by category
Filter by verification status
Add product
Edit product
View product
```

14.2 Product Detail Page

Route:

```
/admin/products/:id
```

Sections:

```
Product identity
Variants
Ingredients
Nutrition
Allergens
Processing level
Decision rules
Lookup history
Audit trail
```

14.3 Product Review Queue

Route:

```
/admin/product-reviews
```

Admin can:

```
Approve match
Change product
Create product
Merge duplicate
Reject
```

Done Criteria

Phase 14 is complete when:

- Admin can add product.
 - Admin can edit product.
 - Admin can verify product.
 - Admin can correct product match.
 - Product lookup improves after correction.
-

17. Phase 15 — Privacy Center

Goal

Build user trust and comply with sensitive data expectations.

Tasks

15.1 Telegram Privacy Command

Command:

```
/privacy
```

Bot shows:

```
Voice audio: deleted after transcription by default  
Transcript: stored for your timeline  
Photos: stored only if needed  
Partner sharing: off by default
```

Buttons:

```
Export my data  
Delete my data  
Change settings  
Privacy policy
```

15.2 Consent Settings

Allow user to control:

```
store_transcripts  
store_raw_audio  
store_photos  
allow_product_personalization  
allow_behavior_insights  
allow_aggregated_analytics
```

```
allow_research  
allow_partner_sharing
```

15.3 Privacy Requests

Create:

```
privacy_requests
```

Types:

```
export_data  
delete_data  
delete_account  
change_consent
```

15.4 Admin Privacy Requests

Route:

```
/admin/privacy-requests
```

Admin can process export/delete.

Done Criteria

Phase 15 is complete when:

- User can view privacy settings.
 - User can request export.
 - User can request deletion.
 - Admin can process requests.
 - Partner sharing defaults to false.
 - Raw audio deletion is visible and auditable.
-

18. Phase 16 — Photo, Receipt, and Restaurant MVP

Goal

Add image-based features only after voice/text/product/reminders work.

Tasks

16.1 Photo Upload

User sends food/product photo.

Bot immediately replies:

Photo saved. I'll analyze it in the background. For faster help, you can also speak or type the product name.

16.2 Receipt Scan

User sends grocery receipt.

System:

- OCR receipt.
- Extract items.
- Categorize basket.
- Suggest missing categories.

16.3 Restaurant Menu Analysis

User sends menu photo/link.

System:

- Extract menu items.
 - Identify better options.
 - Suggest what to order/avoid.
-

Done Criteria

Phase 16 is complete when:

- Photo upload does not block user.
 - Receipt OCR creates grocery record.
 - Menu analysis returns recommendation.
 - Failed photo/OCR cases route to admin review.
-

19. Phase 17 — UI Polish and Responsive Design

Goal

Make the product feel coherent and usable.

Tasks

17.1 Admin UI Polish

Apply design brief:

- Light mode first.
 - Calm blue/green palette.
 - Rounded cards.
 - Clean tables.
 - Right-side drawers.
 - Soft badges.
 - Good empty states.
-

17.2 Telegram Message Polish

Make all messages:

- Short
 - Clear
 - No-shame
 - One next action
 - Button-based where useful
-

17.3 Responsive Admin

Mobile behavior:

- Sidebar becomes hamburger.
 - Tables become cards.
 - Drawers become full-screen sheets.
 - Filters collapse.
-

Done Criteria

Phase 17 is complete when:

- Admin dashboard looks consistent.
 - Telegram copy is polished.
 - Empty/loading/error states exist.
 - Mobile admin is usable enough.
 - No scary medical or insurance-style UI exists.
-

20. Phase 18 — Testing and Edge Cases

Goal

Prevent broken user journeys.

Tasks

18.1 Unit Tests

Test:

```
Meal inference
Intent classifier
Reminder parser
Product matcher
Voice provider router
Safety guardrails
Weekly report generator
```

18.2 Integration Tests

Test:

Telegram webhook
Text food logging
Voice note processing
Product lookup
Reminder creation
Reminder check-in
Weekly report generation
Admin review correction
Privacy request

18.3 Manual Test Scenarios

Test these real scenarios:

Scenario	Expected Result
User says "I ate rice dal curd"	Food log created
User sends voice food log	Voice transcribed and logged
User says "Tony Garden Sunflower"	Product lookup reply
Product not found	General advice + admin review
User says "Remind me daily..."	Reminder created
User taps Taken	Reminder event completed
User sends unclear voice	Ask to type/send again
User sends photo in poor network	Background processing
User asks medical diagnosis	Safe refusal/general wellness response
User asks delete data	Privacy request created

18.4 Safety Tests

Bot must not say:

You will get diabetes.
You have sugar.

```
Stop your medicine.  
This will cure you.  
Insurance company should target you.
```

Allowed:

```
This pattern may not be ideal.  
Consider reducing sweets.  
Speak to a qualified professional for medical advice.  
I can help you track this pattern.
```

Done Criteria

Phase 18 is complete when:

- All core flows pass manually.
- Unit tests pass.
- Integration tests pass.
- Safety tests pass.
- Known errors show human-friendly messages.
- Low-confidence outputs route to review.

21. Phase 19 — Deployment

Goal

Deploy staging and production safely.

Tasks

19.1 Staging Environment

Set up:

```
Vercel staging  
Supabase staging  
Worker staging  
Telegram staging bot
```

```
Redis staging
ASR staging service
```

19.2 Production Environment

Set up:

```
Vercel production
Supabase production
Worker production
Telegram production bot
Redis production
ASR production service
Sentry production
```

19.3 Webhook Configuration

Set Telegram webhook:

```
TELEGRAM_WEBHOOK_URL
TELEGRAM_WEBHOOK_SECRET
```

Verify:

- Webhook receives updates.
 - Secret validation works.
 - Bot replies in production.
-

19.4 Database Migration Deployment

Run migrations in order.

Check:

- Tables exist.
 - RLS enabled.
 - Buckets private.
 - Admin user exists.
 - Service keys server-only.
-

19.5 Smoke Test

Run full smoke test:

1. Start Telegram bot.
 2. Complete onboarding.
 3. Send text food log.
 4. Send voice food log.
 5. Check product.
 6. Create Isabgol reminder.
 7. Tap Taken.
 8. Generate weekly report.
 9. Login admin.
 10. Review event.
 11. Delete raw audio check.
-

Done Criteria

Phase 19 is complete when:

- Production app is live.
 - Telegram bot works.
 - Admin dashboard works.
 - Supabase production schema is correct.
 - Workers are running.
 - ASR works or fallback works.
 - Monitoring is active.
 - Privacy deletion/audio cleanup works.
-

22. Phase 20 — First 50 User Pilot

Goal

Validate the real product before expanding.

Pilot Group

Use 50 users.

Prefer:

- Busy working professionals
- People who use Telegram

- People who shop at Reliance/D-Mart
 - People who eat outside
 - People who want reminders like Isabgol/water
 - People comfortable sending voice notes
-

Pilot Metrics

Track:

Metric	Target
Onboarding completion	70%+
First voice log	50%+
First product lookup	40%+
First reminder created	30%+
Logs per user/week	10+
Reminder completion	50%+
Weekly report open rate	60%+
Day 7 retention	35%+
Wrong AI correction rate	Under 10%
Voice transcription success	95%+

Pilot Questions

Ask users:

1. Was voice easier than typing/photo?
 2. Did the product help at grocery/restaurant?
 3. Was the recommendation useful?
 4. Did reminders feel helpful or annoying?
 5. Did weekly report make sense?
 6. Did you trust the product?
 7. What would make you use it daily?
-

Done Criteria

Pilot is successful if:

- Users send voice/text logs without explanation.
 - Users create reminders.
 - Users check products.
 - Weekly reports are opened.
 - Users return after 7 days.
 - No major privacy/trust issue appears.
 - Admin review workload is manageable.
-

23. Claude/Codex Build Prompts

Prompt 1 — Project Setup

We are building NowWise, a Telegram-first and voice-first AI lifestyle operating system.

Use the approved TRD and create the monorepo structure:

```
apps/web
services/telegram-bot
services/voice-service
services/workers
packages/db
packages/ai
packages/voice
packages/shared
docs
```

Use Next.js with TypeScript and Tailwind for apps/web.

Use Python FastAPI for services/voice-service.

Use Supabase Postgres.

Create .env.example with all required variables.

Do not implement features yet. Only project foundation, tooling, structure, and health checks.

Prompt 2 — Database Schema

Implement the approved NowWise backend schema.

Create ordered Supabase migrations:

```
001_enable_extensions
002_create_enums
003_create_users_and_profiles
004_create_user_events
005_create_voice_events
006_create_food_and_water_logs
007_create_products_and_variants
008_create_product_logs_and_corrections
009_create_restaurant_and_receipt_logs
010_create_reminders
011_create_behavior_patterns_and_reports
012_create_admin_reviews
013_create_consent_privacy_audit
014_create_indexes
015_enable_rls
016_create_storage_buckets
017_create_functions_and_triggers
```

Do not skip RLS.

Do not expose service role key.

Add seed data for local testing.

Prompt 3 — Telegram Foundation

Build the Telegram bot foundation.

Implement `/api/telegram/webhook`.

Verify `TELEGRAM_WEBHOOK_SECRET`.

Create or fetch Telegram user from users table.

Create `user_profiles` and `consent_settings` for new users.

Implement `/start`, `/help`, `/log_food`, `/check_product`, `/set_reminder`, `/my_week`, `/privacy`.

Return quick acknowledgement for heavy tasks.

Do not process voice/photo synchronously inside webhook.

Prompt 4 — Event Engine

Build the universal NowWise event engine.

Every meaningful Telegram interaction should create a `user_events` row.
Implement `createUserEvent`, `updateUserEventStatus`, `markNeedsReview`, `markFailed`, `getUserTimeline`.

Add automatic `occurred_at`, `timezone`, `source`, `event_type`, `status`, `confidence_score`.

Add meal inference helper.

Do not ask users to manually select date/time unless they mention past/future time.

Prompt 5 — Voice Pipeline

Build Telegram voice processing.

When Telegram voice note arrives:

1. Create `user_event`.
2. Create `voice_event`.
3. Queue `processVoiceNote`.
4. Download Telegram audio.
5. Store temporarily in voice-temp bucket.
6. Convert OGG/OPUS to WAV using FFmpeg.
7. Send to local ASR endpoint `/v1/audio/transcriptions`.
8. Use cloud fallback if local ASR fails.
9. Store transcript and provider metadata.
10. Delete raw audio by default.
11. Pass transcript to intent classifier.

Do not embed the full OpenWhisper Electron app.
Use an OpenWhisper-inspired local ASR service only.

Prompt 6 — Food/Product/Reminder Intelligence

Build the first NowWise intelligence flows:

1. Text food logging
2. Voice food logging
3. Product lookup

4. Reminder creation
5. Reminder Taken/Later/Skip
6. Weekly report generation

Use AI provider abstraction.
Use confidence scores.
Route low-confidence outputs to admin_reviews.
Keep language non-medical and non-judgmental.

Prompt 7 — Admin Dashboard

Build the admin dashboard using Next.js, Tailwind, and shadcn/ui.

Pages:

/admin/dashboard
/admin/users
/admin/events
/admin/voice-events
/admin/products
/admin/product-reviews
/admin/reminders
/admin/weekly-reports
/admin/privacy-requests
/admin/settings

Use left sidebar, top filters, tables, badges, and right-side drawers.
Admin must be protected by Supabase Auth and role checks.

24. Build Priority Checklist

Use this checklist.

- [] Repo setup
- [] Environment variables
- [] Supabase project
- [] Database migrations
- [] RLS policies
- [] Storage buckets
- [] Admin auth
- [] Admin layout
- [] Telegram webhook

```
[ ] User onboarding
[ ] Event engine
[ ] Text food logging
[ ] Voice ingestion
[ ] Voice transcription
[ ] Intent classifier
[ ] Product lookup
[ ] Reminder engine
[ ] Reminder buttons
[ ] Weekly report
[ ] Admin review queue
[ ] Product compliance portal
[ ] Privacy center
[ ] Testing
[ ] Staging deployment
[ ] Production deployment
[ ] First 50-user pilot
```

25. Final Done Criteria

NowWise MVP is finished only when all these are true:

User-Side Done

- User can open Telegram bot.
- User can complete onboarding.
- User can send text food log.
- User can send voice food log.
- User can ask product question.
- User can create Isabgol reminder.
- User can tap Taken/Later/Skip.
- User can receive weekly report.
- User can access privacy controls.
- User never needs to manually select date/time for normal logs.

AI/Intelligence Done

- Intent classifier handles food, product, reminder, correction, privacy.
- Food entities are extracted.
- Product matching works with fuzzy search.
- Reminder parser extracts time/frequency.
- Weekly report summarizes behavior.
- Low-confidence events route to admin review.

- Medical diagnosis claims are blocked.
-

Admin Done

- Admin can login.
 - Admin can view users.
 - Admin can view event timeline.
 - Admin can review voice events.
 - Admin can manage products.
 - Admin can correct product matches.
 - Admin can see reminders.
 - Admin can preview weekly reports.
 - Admin can handle privacy requests.
-

Technical Done

- Database migrations run cleanly.
 - RLS is enabled.
 - Raw audio is deleted by default.
 - Storage buckets are private.
 - Telegram webhook is secure.
 - Workers run reliably.
 - ASR has fallback.
 - Monitoring is active.
 - Production deployment works.
-

26. Final Implementation Rule

Do not let the AI agent freelance.

Build in this order:

Foundation → Schema → Auth → Telegram → Event Engine → Voice → Intelligence → Admin → Privacy → Testing → Deploy

The first version of NowWise should prove this:

A user can speak to Telegram, NowWise understands the food/product/reminder, logs it with automatic time, gives a useful recommendation, and builds weekly behavior intelligence.

If that works, the product has a strong foundation.

Everything else — barcode scanner, mobile app, wearables, doctor reports, commerce, and B2B analytics — comes later.